ISM4113 · SYSTEMS DESIGN

Coffee Shop Management System

Midterm Project Report

Student	[Your Name]
Instructor	Dr. Daniel Narváez
Course	ISM4113 – Systems Design
System	Bean & Brew Coffee Shop Management System

A practical demonstration of requirements analysis, data flow, relational data modeling, input design, normalization, and implementation.

Executive Summary

Bean & Brew is a small coffee shop management system designed to organize menu products, customer information, order transactions, and basic sales reporting in one application. It uses a Flask web interface and a relational SQLite database. The project demonstrates how systems analysis concepts become requirements, process flows, data structures, input controls, and implementation decisions.

A. System Overview and Requirements

Problem and purpose. A small coffee shop may record menu items and sales manually, causing repeated data entry, inconsistent prices, calculation mistakes, and difficulty identifying popular products. Bean & Brew centralizes product, customer, and order records and calculates totals from saved order lines.

Intended users. Shop staff or a manager maintain menu information, optionally register customers, record sales, and review summaries. Walk-in orders are supported.

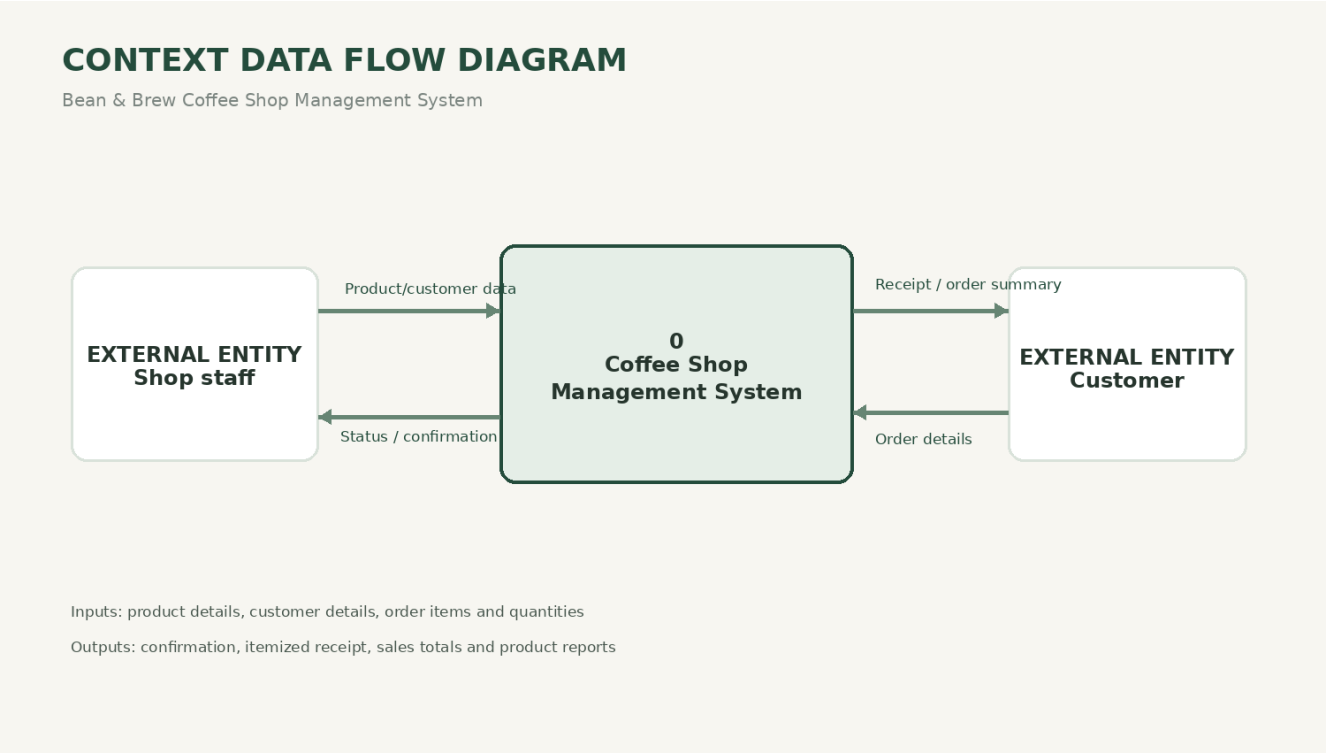
Main functions. Dashboard; product catalogue; customer directory; multi-item order entry; itemized receipt; daily sales and top-selling product reports.

Requirements traceability

Requirement	Design decision	Implementation evidence
Add products	Required name/category and non-negative price	products.html; products table
Store customers	Separate customer table; unique email when supplied	customers.html; customers table
Record multi-item sale	Order header and separate order-item rows	orders and order_items
Support walk-ins	Customer foreign key is nullable	new_order.html; schema
Preserve sale totals	Store price charged on each order line	order_items.unit_price
Protect consistency	Keys, constraints, validation, rollback	schema.sql; routes.py
Support decisions	Daily sales and best-seller summaries	/reports route

B. Data Flow Design

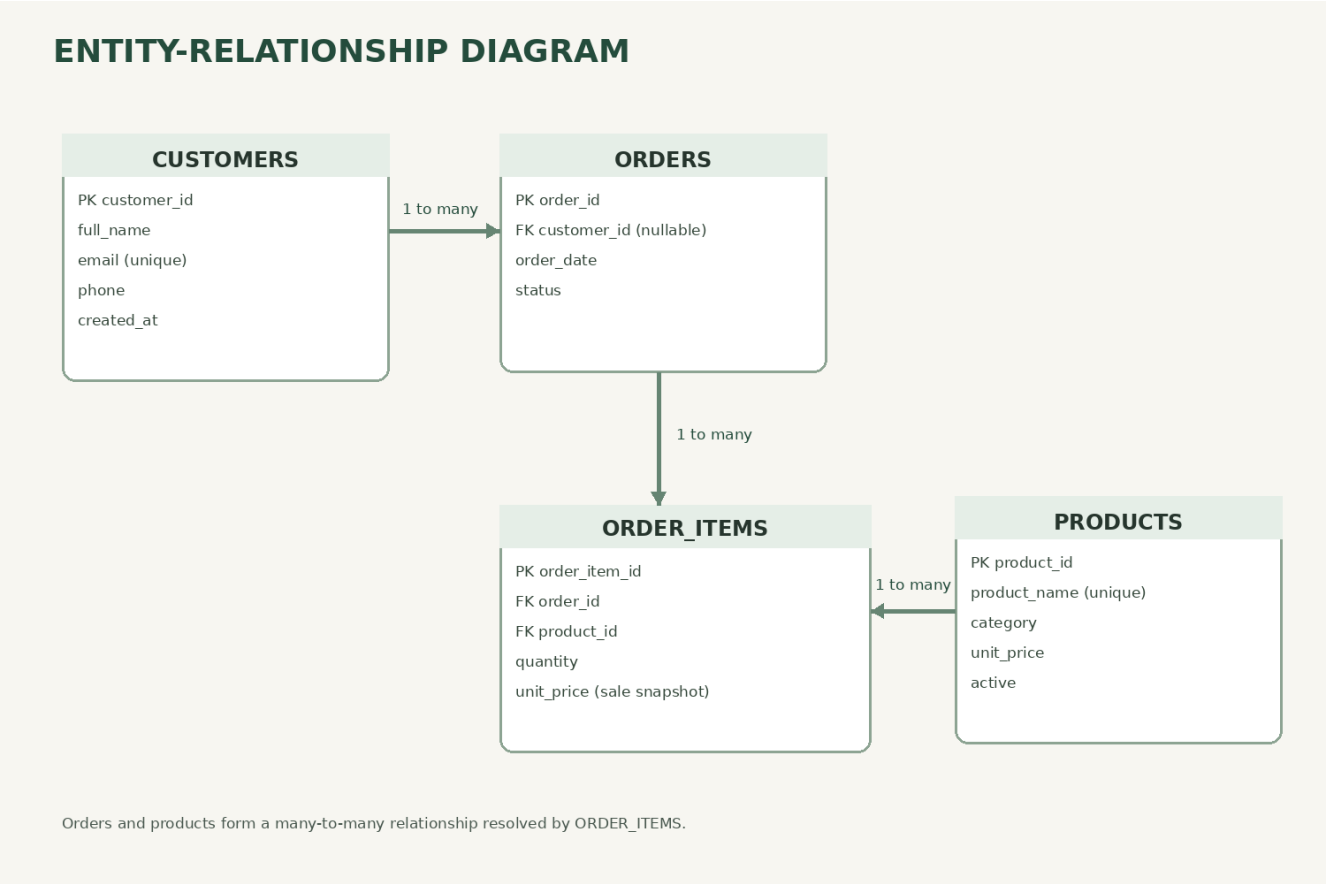
The context DFD models the application as one process. Shop staff provide product, customer, and order information. The system processes inputs, stores records, and returns confirmations, receipts, and sales summaries. Customers provide order details and receive receipts. The context view focuses on the system boundary rather than internal database tables.



Important flow: staff select an optional customer, products, and quantities; the application validates values and retrieves product prices; it writes an order header and item rows; database constraints check relationships and values; a successful transaction is committed and displayed as a receipt. If a write fails, the transaction is rolled back.

C. Database Design and Data Dictionary

The ER diagram identifies four entities. Each table has a primary key. **orders.customer_id** references **customers.customer_id** and is nullable to support walk-in customers. **order_items.order_id** references orders, while **order_items.product_id** references products.



Cardinality

• Customer to orders: one-to-many. A customer may have many orders; an order has zero or one registered customer.

• Order to order items: one-to-many. Each order has one or more item rows.

• Product to order items: one-to-many. A product can appear on many order lines.

• Order to product: many-to-many, resolved through the associative entity order_items.

Core fields and keys

Table	Key fields	Purpose
customers	customer_id (PK); email (unique)	Customer name and optional contact details
products	product_id (PK); product_name (unique)	Menu name, category, current price, active flag
orders	order_id (PK); customer_id (FK)	Transaction date, status, optional customer
order_items	order_item_id (PK); order_id / product_id (FK)	Products, quantity, and sale-time unit price

The full field-by-field dictionary is provided in docs/data_dictionary.md. The order-item unit price is a historical snapshot of the price charged at the time of sale; later menu-price changes do not rewrite historical receipts.

D. Normalization and Data Integrity

A single flat transaction table could repeat customer contact details and product descriptions for every item sold. That creates update anomalies: changing a customer's phone number or a product category could require editing many rows, and inconsistent versions could remain.

First Normal Form (1NF). Each field stores one value; multiple products are separate rows rather than a comma-separated list.

Second Normal Form (2NF). Single-column primary keys are used, so non-key attributes describe the row identified by that key rather than part of a composite key.

Third Normal Form (3NF). Customer details and menu attributes are kept in their own tables, avoiding transitive dependencies in sales rows. The order-line price is intentionally retained as a sale-time snapshot for historical accuracy.

Integrity mechanisms

• Primary keys uniquely identify records; foreign keys preserve relationships between records.

• NOT NULL protects required attributes; UNIQUE prevents duplicate product names and duplicate non-null emails.

• CHECK constraints reject negative prices, non-positive quantities, invalid active flags, and unsupported statuses.

• The application validates inputs and uses parameterized SQL.

• Order header and item rows are committed together; failed writes are rolled back.

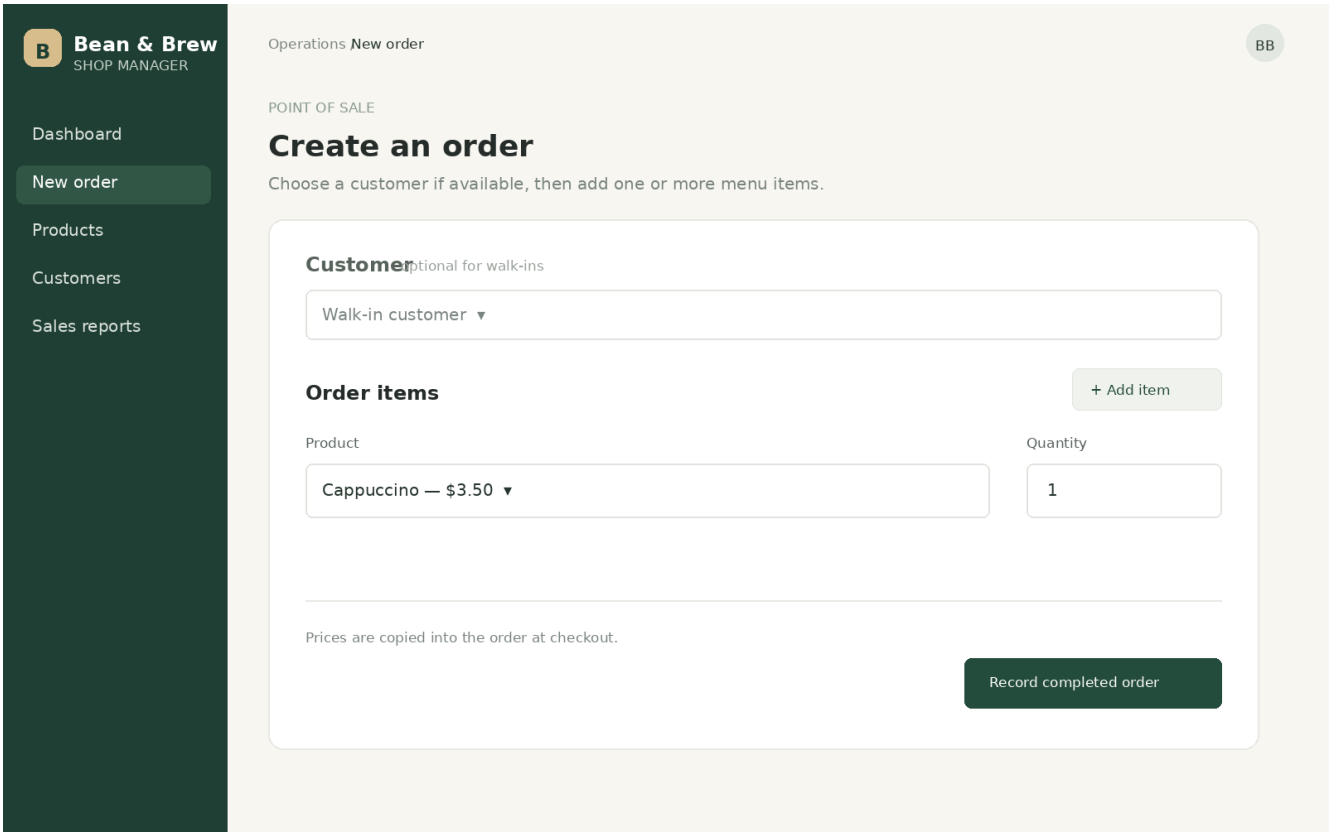
• Foreign-key enforcement is enabled on each SQLite connection.

Example before and after

Flat design (before)	Normalized design (after)
Customer name, phone, product name, category, price repeated on every sale line	Customer details stored once in customers; product details stored once in products
Order containing several products in a single text field	One order row plus one order_items row per purchased product
A price change could alter interpretation of old sales	Current product price is separate from unit price recorded on each order line

E. Input Design

The order form uses a dropdown for the optional customer and product selection, numeric quantity fields with a minimum of one, clear field labels, and an explicit action button. Dropdowns reduce spelling differences, numeric constraints reduce invalid quantities, and labels help staff understand what each field represents. A walk-in option keeps the process efficient when a customer does not need a profile.



The implementation provides success and error feedback after form submission. These choices support effectiveness, accuracy, consistency, simplicity, and ease of use. The image is a design reference; running the application displays the live interface.

F. Connection with Course Theory

This project moves from a practical problem to requirements, process modeling, data modeling, input design, implementation, and verification. The DFD defines the system boundary and major information exchanges. The ER diagram describes entities and cardinality before table implementation. Normalization separates facts by entity to reduce anomalies. Input design guides users toward valid data, while database constraints protect integrity beyond the interface.

Examples of requirement-to-design reasoning

Business need	Design decision	Why it matters
Record multi-item sales	Separate orders from order_items	One transaction can contain multiple products without repeating header fields
Keep historical totals accurate	Store sale-time unit price	Old receipts remain stable when menu prices change
Support walk-in sales	Nullable customer foreign key	Staff can record a sale without collecting customer details
Reduce input errors	Dropdowns, required fields, quantity limits	Encourages consistent, valid entries

Business need	Design decision	Why it matters
Produce useful summaries	Aggregate completed order data	Supports operational decisions about sales and popular products

Limitations and Future Improvements

This is an academic prototype. It does not include user authentication, staff roles, tax or discount rules, inventory depletion, payment processing, receipt printing, cloud hosting, or automated backups. A production release should add authentication and authorization, CSRF protection, audit logging, secure configuration, backup/restore procedures, and additional tests. Currency is displayed in USD for demonstration.

Conclusion

Bean & Brew demonstrates a working small-scale system with a clear relational model, validated input, persistent records, and useful reports. The repository includes source code, diagrams, a data dictionary, requirements traceability, tests, and this report so the design can be reviewed against the course rubric.

References

Flask. (n.d.). *Flask documentation*. https://flask.palletsprojects.com/

SQLite. (n.d.). *SQLite documentation*. https://www.sqlite.org/docs.html

Appendix: Repository verification

Run locally with Python 3.10 or newer: create a virtual environment, install requirements.txt, run python run.py, then open http://127.0.0.1:5000. Run automated tests with python -m unittest discover -s tests -v. The database is initialized automatically and sample products are inserted on first run.

Submission reminder: replace [Your Name], create/push the GitHub repository, verify the public/private access setting, and grant GitHub user jdanieln access if the repository is private.